CEUE203 - OOP25AIML034

PRACTICAL – 10

Aim:

To develop Java multithreading programs using ExecutorService, thread synchronization, wait() and notify(), and object locks to execute tasks using a thread pool, coordinate producer–consumer operations, and demonstrate and prevent deadlocks.

Code Screenshot:

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;

import java.util.concurrent.TimeUnit;


public class Main {

   public static void main(String[] args) {

      ExecutorService executor =

            Executors.newFixedThreadPool(3);


      for (int i = 1; i <= 10; i++) {

         final int taskId = i;


         executor.submit(() -> {

            System.out.println(

                  "Task " + taskId + " executed by "
```

```
            + Thread.currentThread().getName());

        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    });
}

executor.shutdown();

try {
    if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
        executor.shutdownNow();
    }
} catch (InterruptedException e) {
    executor.shutdownNow();
    Thread.currentThread().interrupt();
}

System.out.println("All tasks completed.");
```

```
    }
}
```

2.

```
import java.util.LinkedList;
import java.util.Queue;

class Buffer {
    private final Queue<Integer> queue = new LinkedList<>();
    private final int capacity = 3;

    public synchronized void produce(int item)
            throws InterruptedException {
        while (queue.size() == capacity) {
            wait();
        }

        queue.add(item);
        System.out.println("Produced: " + item);

        notifyAll();
    }
```

```
    public synchronized int consume()
            throws InterruptedException {
        while (queue.isEmpty()) {
            wait();
        }

        int item = queue.remove();
        System.out.println("Consumed: " + item);

        notifyAll();
        return item;
    }
}

public class Main {
    public static void main(String[] args)
            throws InterruptedException {

        Buffer buffer = new Buffer();
        int items = 10;
```

```
Thread producer = new Thread(() -> {
    try {
        for (int i = 1; i <= items; i++) {
            buffer.produce(i);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

Thread consumer = new Thread(() -> {
    try {
        for (int i = 1; i <= items; i++) {
            buffer.consume();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

producer.start();
consumer.start();
```

```
        producer.join();
        consumer.join();

        System.out.println("Production and consumption completed.");
    }
}
```

3.

```
public class Main {

    static final Object lockA = new Object();
    static final Object lockB = new Object();

    public static void main(String[] args)
            throws InterruptedException {

        System.out.println("Deadlock Demonstration:");

        Thread t1 = new Thread(() -> {
            synchronized (lockA) {
                System.out.println("Thread 1 locked A");
```

```
        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }

        synchronized (lockB) {
            System.out.println("Thread 1 locked B");
        }
    }
});

Thread t2 = new Thread(() -> {
    synchronized (lockB) {
        System.out.println("Thread 2 locked B");

        try {
            Thread.sleep(100);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
```

```
            return;
        }

        synchronized (lockA) {
            System.out.println("Thread 2 locked A");
        }
    }
});

t1.setDaemon(true);
t2.setDaemon(true);

t1.start();
t2.start();

t1.join(500);

System.out.println("Deadlock detected: "
        + (t1.getState() == Thread.State.BLOCKED
        && t2.getState() == Thread.State.BLOCKED));

System.out.println("\nDeadlock Prevention:");
```

```
Thread t3 = new Thread(() -> {
   synchronized (lockA) {
      synchronized (lockB) {
         System.out.println(
               "Thread 3 acquired A then B");
      }
   }
});

Thread t4 = new Thread(() -> {
   synchronized (lockA) {
      synchronized (lockB) {
         System.out.println(
               "Thread 4 acquired A then B");
      }
   }
});

t3.start();
t4.start();
```

```
        t3.join();

        t4.join();

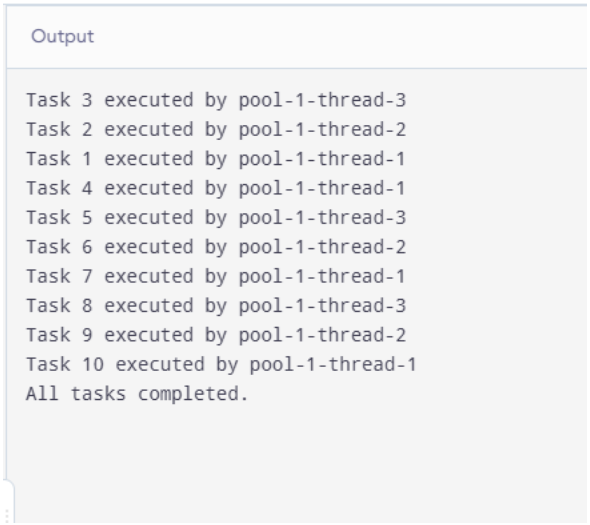


        System.out.println("Both threads completed successfully.");

    }

}
```

Output Screenshot:



Output

```
Produced: 1
Produced: 2
Produced: 3
Consumed: 1
Consumed: 2
Consumed: 3
Produced: 4
Produced: 5
Produced: 6
Consumed: 4
Consumed: 5
Consumed: 6
Produced: 7
Produced: 8
Produced: 9
Consumed: 7
Consumed: 8
Consumed: 9
Produced: 10
Consumed: 10
Production and consumption completed.
```

Output

```
Deadlock Demonstration:
Thread 1 locked A
Thread 2 locked B
Deadlock detected: true

Deadlock Prevention:
```